

Design and Implementation of a Multi-Factor Authentication System Using TOTP

Van Thuong Nguyen

99843952874

Department of Computer Engineering

thuong@ogr.eskisehir.edu.tr

Canay Moradi

15735060434

Department of Computer Engineering

canaymoradi@ogr.eskisehir.edu.tr

Abstract—The goal of this project is to design a more secure system for web application sign-ups and sign-ins. The traditional approach relies only on a username and password, which is vulnerable to attacks such as brute-force attempts, phishing, and data breaches that can expose user credentials. To address this, we implement a multi-factor authentication (MFA) system that combines knowledge-based authentication (password) with possession-based authentication using a time-based one-time password (TOTP). In our system, user credentials are stored securely, and after a successful login attempt, the user must complete a second verification step by scanning a QR code and generating a one-time password on a secondary device. This additional layer improves overall security and reduces the risk of unauthorized access compared to single-factor authentication systems.

Index Terms—Cryptography, Security Analysis, Multi-Factor Authentication, Term Project

I. REPORT REQUIREMENTS

Keep every heading below, but a section may be marked “N/A” if genuinely not applicable to your project.

- 1) **Introduction** — In this project, we aim to improve the security of web application sign-up and login systems by implementing a multi-factor authentication (MFA) approach. Our system combines password-based authentication with a time-based one-time password (TOTP) generated on a separate device. We focus on demonstrating how adding an additional authentication factor can reduce the risk of unauthorized access compared to single-factor systems.
- 2) **Background and Related Work** — Authentication is the process of verifying the identity claimed by a user or system entity [3]. Most web applications rely on single-factor authentication using a username and password. While widely used, this approach is vulnerable to attacks such as brute-force attempts, phishing, and credential leakage from data breaches. To improve security, multi-factor authentication (MFA) requires two or more independent authentication factors, such as something the user knows (password), something the user has (a device), or something the user is (biometric data) [3]. This significantly reduces the risk of unauthorized access

if one factor is compromised. A widely used possession-based method is the Time-Based One-Time Password (TOTP) algorithm, which generates a short-lived code based on a shared secret and the current time [4]. This ensures that each authentication code is valid only for a limited duration. Due to the weaknesses of password-only systems, MFA has become a standard approach in modern security practices. Our project follows this direction by combining password-based authentication with TOTP verification to enhance overall system security. In terms of threat models, we consider attackers who may attempt to gain unauthorized access through methods such as brute-force attacks, phishing, and credential stuffing. A brute-force attacker repeatedly tries different password combinations. Phishing attacks trick users into revealing their credentials. Credential stuffing exploits reused passwords leaked from previous data breaches. In this project, we assume the attacker may obtain a user's password but does not have access to the user's secondary authentication device, which motivates the use of multi-factor authentication. About related work, MFA has been widely adopted in modern systems such as online banking, email services, and cloud platforms. Many of these systems implement Time-Based One-Time Password (TOTP) mechanisms through authenticator applications like Google Authenticator (what we are using in this project) or Microsoft Authenticator. These implementations demonstrate that combining password-based authentication with a second factor significantly improves security in practice. Our project follows a similar approach by integrating TOTP-based verification into a web authentication system.

3) **Proposed Methodology — Design Goals:**

The main goal of this project is to design and implement a practical Multi-Factor Authentication (MFA) system that improves the security of traditional password-based login mechanisms. The system is designed to satisfy several security and performance objectives.

First, the user's password should be securely hashed before being stored in the database. Second, authentication should not rely only on passwords, since passwords can

be stolen, guessed, reused, or obtained through phishing. Therefore, the system combines three authentication factors: password verification, time-based one-time password (TOTP) verification, and simulated biometric verification [5].

From a performance perspective, the system should remain lightweight and usable. The authentication process should not introduce excessive delay during login, OTP verification, or JWT generation. The system should also be simple enough to run in a local development environment while still demonstrating the core cryptographic and security principles behind MFA.

Threat Model:

The system assumes an attacker who may attempt to gain unauthorized access by guessing passwords, reusing leaked credentials, brute force technique, or performing phishing-based attacks. The attacker may know the user's email address and may attempt multiple login attempts. The attacker may also try to replay an old OTP code or access a protected page without a valid session token.

However, the system assumes that the attacker does not have access to the user's authenticator application or device that generates the TOTP code. The system also assumes that the backend server and database environment are trusted during the experiment. Side-channel attacks, physical attacks, and quantum attacks are outside the scope of this project because the implementation focuses on practical web authentication rather than low-level cryptographic hardware security.

Construction or Attack Strategy:



Fig. 1. Overall System Architecture

The proposed system consists of three main components: a React frontend, a FastAPI backend, and a PostgreSQL database. The frontend provides the user interface for registration, login, OTP verification, and dashboard access. The backend handles authentication logic, password hashing, OTP validation, JWT generation, and database communication. The database stores user accounts, MFA factors, login attempts, and authentication session information.

During registration, the user enters an email address and password. The backend hashes the password before storing it in the database. At the same time, a TOTP secret is generated and stored as an MFA factor. The backend then creates a QR code that can be scanned using an authenticator application.



Fig. 2. Multi-Factor Authentication Flow

During login, the user first submits the email and password. If the password is valid, the system proceeds to the MFA stage. The user then enters the OTP code generated by the authenticator application and completes a simulated biometric verification step. If both checks are successful, the backend generates a JSON Web Token (JWT), which is returned to the frontend and used to access protected resources such as the dashboard.

Evaluation Metrics:

The system is evaluated using both security and performance metrics. From the security perspective, the evaluation focuses on resistance against common authentication attacks such as brute-force login attempts, credential reuse, replay attacks, and unauthorized dashboard access. Password security is evaluated based on whether passwords are hashed before storage. OTP security is evaluated based on whether time-sensitive TOTP codes are required after password verification. Session security is evaluated by checking whether protected routes require a valid JWT token.

From the performance perspective, the system is evaluated by measuring the execution time of the main authentication stages. These include password verification time, OTP verification time, JWT generation time, and total MFA authentication time. The measured values are recorded in milliseconds and compared to determine whether the additional security layers introduce acceptable overhead.

Experimental Setup:

The system was developed and tested in a local development environment. The backend was implemented using Python with the FastAPI framework. PostgreSQL was used as the database management system. The frontend was implemented using React and connected to the backend through HTTP API requests. The main software libraries used in the backend include bcrypt for password hashing, pyotp for TOTP generation and verification, qrcode for QR code generation, SQLAlchemy for database interaction, and python-jose for JWT handling.

The experiment was conducted on a local laptop environment. Since the project does not use a machine learning dataset, no external dataset was required. The test data consisted of manually created user accounts and generated OTP values from an authenticator application. Random values, such as TOTP secrets and UUID identifiers, were generated using the built-in random generation mechanisms provided by the selected libraries and PostgreSQL.

4) Security & Performance Evaluation —

The proposed MFA system was evaluated using both security and performance metrics. From the security perspective, the system was tested against several common authentication-related attack scenarios, including incorrect password attempts, invalid OTP input, replaying expired OTP codes, unauthorized access without a valid token, and modified JWT tokens. These tests were conducted to verify whether each authentication layer works as expected and whether the system correctly

prevents unauthorized access.

Table I summarizes the security test scenarios used in the evaluation.

TABLE I
SECURITY TEST SCENARIOS

Test	Scenario	Expected Result	Result
Wrong password	Incorrect password is entered.	Login is rejected.	Passed
Wrong OTP	Correct password but invalid OTP is entered.	MFA fails.	Passed
Expired OTP	Old OTP code is reused.	OTP is rejected.	Passed
No JWT	Dashboard is accessed without token.	Access is denied.	Passed
Modified JWT	JWT token is manually changed.	Token is rejected.	Passed

The results show that the system correctly rejects invalid authentication attempts at different stages of the login process. A user cannot access the dashboard using only a password, because OTP verification is required after the first authentication step. Similarly, expired OTP codes cannot be reused because TOTP values are time-sensitive. Protected routes also require a valid JWT token, which prevents unauthorized access after the login process.

In addition to the test scenarios, Table II summarizes the assumed threats, implemented defenses, and remaining limitations of the proposed system.

TABLE II
THREAT MODEL AND IMPLEMENTED DEFENSES

Threat	Defense Used	Limitation
Brute-force password attack	bcrypt password hashing increases the computational cost of password guessing.	Rate limiting is still needed for stronger protection.
Credential stuffing	TOTP adds a second authentication factor after password verification.	The system may still be vulnerable if the attacker also obtains the OTP.
OTP replay attack	TOTP codes are time-sensitive and expire after a short period.	Correct time synchronization is required.
Unauthorized dashboard access	Protected routes require a valid JWT token.	JWT expiration and secure storage must be configured properly.
Phishing attack	MFA reduces the risk of access using only stolen passwords.	Real-time phishing can still capture both password and OTP.

Although the system improves security against password leakage and credential reuse, it does not fully prevent real-time phishing attacks where an attacker may capture both the password and the OTP code. In addition, features such as account lockout, rate limiting, backup recovery codes, and phishing-resistant authentication methods such as WebAuthn/FIDO2 are not included in the current prototype. Therefore, the system should be understood as a practical MFA prototype rather than a complete production-ready authentication system.

From the performance perspective, the system was evaluated by measuring the execution time of each authentication stage. The results are summarized in Table III. Password verification required approximately 291.57 ms, which is expected due to the computational cost of hashing algorithms such as bcrypt. OTP verification was significantly faster, taking around 6.78 ms on average, while JWT generation introduced negligible overhead at approximately 0.45 ms. The total authentication time was measured to be approximately 298.8 ms.

TABLE III
AUTHENTICATION PERFORMANCE

Stage	Avg Time (ms)
Password Verification	291.57
OTP Verification	6.78
JWT Generation	0.45
Total MFA	298.80

The reported values represent the average of 10 repeated authentication trials. The result from Table III shows that password verification is the most expensive stage because bcrypt is intentionally designed to be computationally costly. This is useful for security because it slows down brute-force attacks. OTP verification and JWT generation are much faster, meaning that the second factor does not create significant performance overhead. Since the total MFA time is below one second, the system remains usable for normal web login scenarios. These results indicate that the proposed MFA system introduces minimal additional delay while significantly improving authentication security. The overhead remains acceptable for real-world applications, demonstrating that enhanced security can be achieved without sacrificing system performance.

5) Implementation and Code Explanation —

The system is implemented using a modular architecture consisting of a backend, a frontend, and a database layer. The backend is developed using FastAPI in Python, which handles authentication logic, OTP verification, and session management. The frontend is implemented using React, providing user interfaces for registration, login, OTP verification, and access to protected resources. PostgreSQL is used as the database system to store user accounts, authentication factors, and session-related information.

The backend is organized into several main modules. The `main.py` file defines API endpoints such as registration, login, OTP verification, and protected routes. The `auth.py` module contains utility functions for password hashing, OTP generation and validation, and JWT creation. The `models.py` file defines database schemas using SQLAlchemy, including user accounts and MFA-related data. The `database.py` module manages database connections and sessions.

The frontend is structured into multiple components corresponding to each stage of the authentication process. These include pages for user registration, login, OTP verification, and dashboard access. Communication

between the frontend and backend is handled through HTTP API requests using Axios.

To reproduce the system, the backend server must be started using FastAPI, and the frontend application must be run using a Node.js development environment. The database must be initialized with the provided schema. Detailed instructions for running the system are included in the README file, ensuring that the project can be executed on a clean environment without requiring access to the original development setup.

Column	Description
email	User email
password_hash	Hashed password (bcrypt)

- 6) **Personal Contributions** — The project was completed with a division of tasks between both members. Canay was responsible for designing and implementing the database, including securely storing user credentials and connecting it to the authentication system. Thuong worked on developing the web application, including both the front-end and back-end parts for user registration and login. The implementation of the TOTP system using Google Authenticator was done together. Both members contributed to integrating it into the system, testing, and fixing issues. Both members also worked together on evaluating the system and writing the final report.

- 7) **Discussion and Conclusion** — The results of this project show that adding a second authentication factor significantly improves the overall security of a web application compared to using only a username and password. By integrating TOTP-based verification, the system reduces the risk of unauthorized access even if user credentials are compromised. However, there are some limitations to our approach. The system depends on the user having access to a second device, that may not always be convenient. Also, our implementation does not consider more advanced threats such as phishing attacks that can capture both the password and the one-time code in real time. For future work, the system could be improved by adding stronger protections such as phishing-resistant authentication methods, backup recovery options, and account lockout or rate-limiting mechanisms. Further testing on performance and usability with more users could also help evaluate how well the system works in real world scenarios.

- 8) **Ethical Statement** — This project was conducted in accordance with academic honesty guidelines, and all work presented is original. Any external sources used have been properly cited. No unauthorized collaboration or plagiarism was involved in the development of this project. The system was developed and tested in a controlled environment, and no real user data was collected. Any credentials used during testing were generated specifically for this project. The project does not involve the handling of sensitive personal data beyond what is necessary for demonstrating the authentication process.

- 9) **AI Tools Usage** — AI-based tools (LLM) were used to assist during different stages of this project. They

were mainly used for debugging support, clarifying programming related issues, and helping identify relevant academic resources. In addition, AI tools were used to improve the clarity and structure of the report, to make it more consistent with academic writing standards. Core design decisions, implementation, and evaluation of the system were done by project members. The use of AI was limited to assistance and did not replace our own understanding or work.

Additionally, we also using Overleaf website for improve our collaborating workload for writing report in Latex format. It helps us to work remotely but still can check the progress of the whole project progress.

- 10) **References** — cite all sources in IEEE style; include standard identifiers (DOI, URL) when available.

All reports **must** use this IEEE journal L^AT_EX template. Submissions in any other format will not be graded.

II. SUBMISSION DETAILS

A. Project Timeline

Milestone	Deadline
Project Selection	09.03.2026, 23:59
Progress Report Submission	05.04.2026, 23:59
Final Report Submission	10.05.2026, 23:59
Project Presentations	14.05.2026

TABLE IV
PROJECT TIMELINE

B. Late Submission Penalties

- 1) **Final Report:** 5 points deducted for every calendar day submitted after 27.05.2025.

C. Files to be Sent

- 1) All source code, build/run scripts, datasets, and test vectors needed for replication.
- 2) A concise README explaining how to compile, execute, and verify results on a fresh system.

III. EVALUATION CRITERIA

- 1) **Progress Report (20 %)** — assesses clarity of goals, feasibility of plan, and timely progress.
- 2) **Final Report (80 %)** - (i) Technical soundness of proofs or empirical methodology, (ii) Implementation quality, efficiency, and reproducibility, (iii) Depth and clarity of analysis, writing style, and adherence to template.

REFERENCES

- [1] R. Cramer and V. Shoup, "A Practical Public-Key Cryptosystem Provably Secure Against Adaptive Chosen-Ciphertext Attack," *Journal of Cryptology*, vol. 12, no. 4, pp. 265–292, 1999.
- [2] National Institute of Standards and Technology, "FIPS 197: Advanced Encryption Standard," 2001. Available: <https://csrc.nist.gov/publications/fips/fips197/fips-197.pdf>

- [3] National Institute of Standards and Technology (NIST), "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, 2017.
- [4] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, Internet Engineering Task Force (IETF), 2011
- [5] V. Venukumar and V. Pathari, "Multi-factor authentication using threshold cryptography," 2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Jaipur, India, 2016, pp. 1694-1698, doi: 10.1109/ICACCI.2016.7732291.
- [6] D. K. Vakharia and J. Banerjee, "Enhancing Two-Factor Authentication through MAC Address Verification: A Pythonbased Security Approach," 2025 4th International Conference on Innovative Mechanisms for Industry Applications (ICIMIA), Tirupur, India, 2025, pp. 1685-1689, doi: 10.1109/ICIMIA67127.2025.11200919.
- [7] B. O. ALSaleem and A. I. Alshoshan, "Multi-Factor Authentication to Systems Login," 2021 National Computing Colleges Conference (NCCC), Taif, Saudi Arabia, 2021, pp. 1-4, doi:10.1109/NCCC49330.2021.9428806.
- [8] A. Mansour, M. Sadik and E. Sabir, "Multi-factor authentication based on multimodal biometrics (MFA-MB) for Cloud Computing," 2015 IEEE/ACS 12th International Conference of Computer Systems and Applications (AICCSA), Marrakech, Morocco, 2015, pp. 1-4, doi: 10.1109/AICCSA.2015.7507257.